PYTHON PROGRAMMING

Algorithms, Flowcharts & Pseudocode

Name	B. Akhil Kumar
Section	CSM
Roll Number	2600068

Ten problem-solving programs for academic practice. Each problem includes an aim, step-by-step algorithm, flowchart, and language-independent pseudocode.

Note: Rates, eligibility thresholds, deductions, concessions, and fee policies are sample values for learning. Replace them with the exact rules provided by your instructor.

1. University Scholarship Allocation

Aim: Evaluate applicants using aggregate marks, minimum subject marks, attendance, and family income; rank eligible students with tie-breaking rules.

Algorithm

1. Start.

2. Input the number of applicants and each applicant's name, aggregate marks, minimum subject marks, attendance, and annual family income.

3. Check eligibility: aggregate marks >= 75, minimum subject marks >= 60, attendance >= 75%, and annual income <= ■250,000.

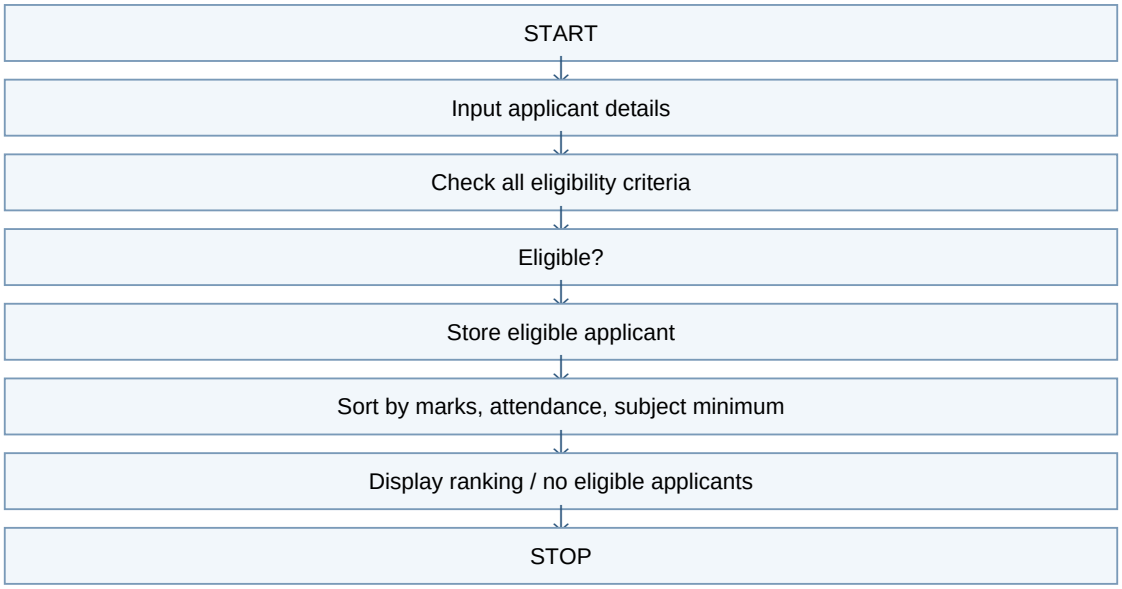
4. Store applicants who satisfy every condition.

5. Sort eligible applicants by aggregate marks, then attendance, then minimum subject marks, all in descending order.

6. Display the ranked list. If no applicant qualifies, display a suitable message.

7. Stop.

Flowchart



Pseudocode

```
BEGIN
    INPUT number_of_applicants
    eligible_list = empty list

    FOR each applicant
        INPUT name, aggregate, subject_min, attendance, income
        IF aggregate >= 75 AND subject_min >= 60
           AND attendance >= 75 AND income <= 250000 THEN
            ADD (name, aggregate, attendance, subject_min)
                TO eligible_list
        ENDIF
    ENDFOR

    SORT eligible_list BY aggregate DESC,
         attendance DESC, subject_min DESC
    DISPLAY eligible_list
END
```

2. Progressive Electricity Billing

Aim: Calculate monthly electricity bills using progressive slabs, fixed charges, surcharges, and rebates; identify unusually large increases.

Algorithm

1. Start.

2. Input the number of months and the units consumed each month.

3. Calculate energy charges progressively: first 100 units at ■2/unit; next 100 at ■3/unit; next 100 at ■5/unit; remaining units at ■7/unit.

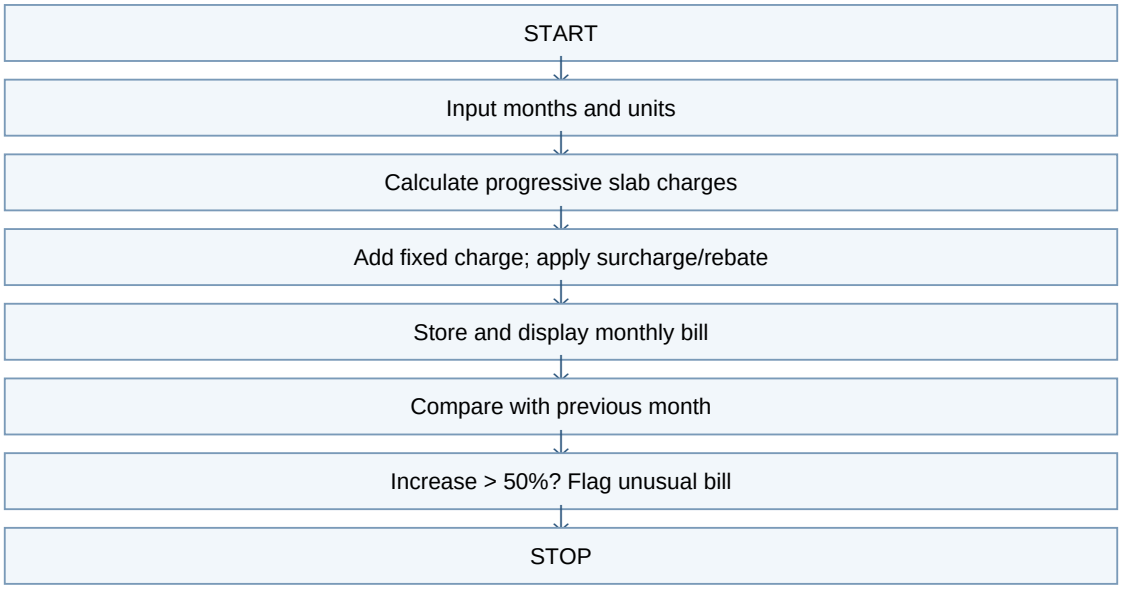
4. Add fixed charge of ■50. Add a 10% surcharge on energy charges if usage exceeds 300 units. Apply a 5% rebate on energy charges if usage is below 100 units.

5. Store and display each monthly bill.

6. Compare each bill with the previous month's bill. If the increase exceeds 50% of the previous bill, flag it as unusual.

7. Stop.

Flowchart



Pseudocode

```
BEGIN
    INPUT months
    FOR month = 1 TO months
        INPUT units
        energy = 0
        IF units > 300 THEN
            energy = 200 + 300 + 500 + (units - 300) * 7
        ELSE IF units > 200 THEN
            energy = 200 + 300 + (units - 200) * 5
        ELSE IF units > 100 THEN
            energy = 200 + (units - 100) * 3
        ELSE
            energy = units * 2
        ENDIF

        surcharge = 0
        rebate = 0
        IF units > 300 THEN surcharge = energy * 0.10
        IF units < 100 THEN rebate = energy * 0.05
        bill[month] = energy + 50 + surcharge - rebate
        DISPLAY bill[month]
    ENDFOR

    FOR month = 2 TO months
```

```
        IF bill[month] > bill[month-1] * 1.50 THEN
            DISPLAY "Unusual increase", month
        ENDIF
    ENDFOR
END
```

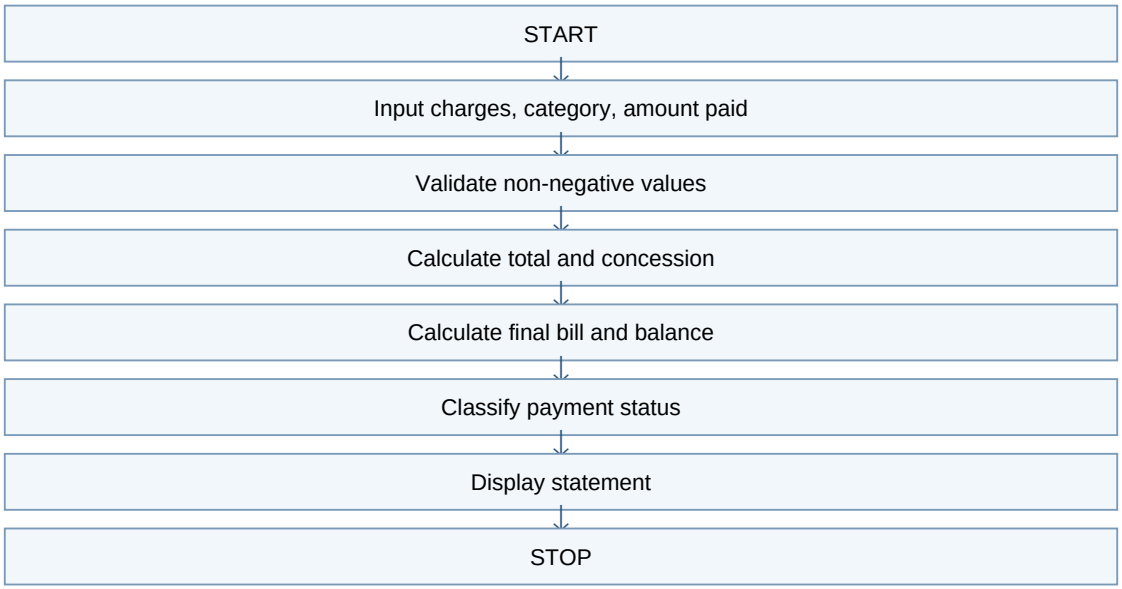
3. Hospital Billing and Patient Classification

Aim: Calculate hospital charges, apply a category-based concession, validate payments, and classify payment status.

Algorithm

1. Start.
2. Input consultation, diagnostic, and treatment charges; patient category; and amount paid.
3. Reject negative charges or negative payments.
4. Add all charges to get the total. Apply a 10% concession for senior patients; otherwise, apply no concession.
5. Calculate the final bill after concession.
6. If payment exceeds the final bill, report invalid/excess payment. Otherwise calculate the balance.
7. Classify the account as Fully Paid, Partially Paid, or Unpaid.
8. Display the bill and stop.

Flowchart



Pseudocode

```
BEGIN
    INPUT consultation, diagnostics, treatment
    INPUT category, paid

    IF any charge < 0 OR paid < 0 THEN
        DISPLAY "Invalid input"
    ELSE
        total = consultation + diagnostics + treatment
        IF category = "senior" THEN
            concession = total * 0.10
        ELSE
            concession = 0
        ENDIF
        final_bill = total - concession

        IF paid > final_bill THEN
            DISPLAY "Payment exceeds bill"
        ELSE
            balance = final_bill - paid
            IF balance = 0 THEN
                status = "Fully Paid"
            ELSE IF paid > 0 THEN
                status = "Partially Paid"
            ELSE
                status = "Unpaid"
```

```
            ENDIF
            DISPLAY final_bill, balance, status
        ENDIF
    ENDIF
END
```

4. Employee Payroll and Deduction Analysis

Aim: Calculate gross salary from pay components, apply progressive deductions, and print a salary statement.

Algorithm

1. Start.

2. Input basic pay, allowances, overtime payment, and incentives.

3. Validate that all components are non-negative.

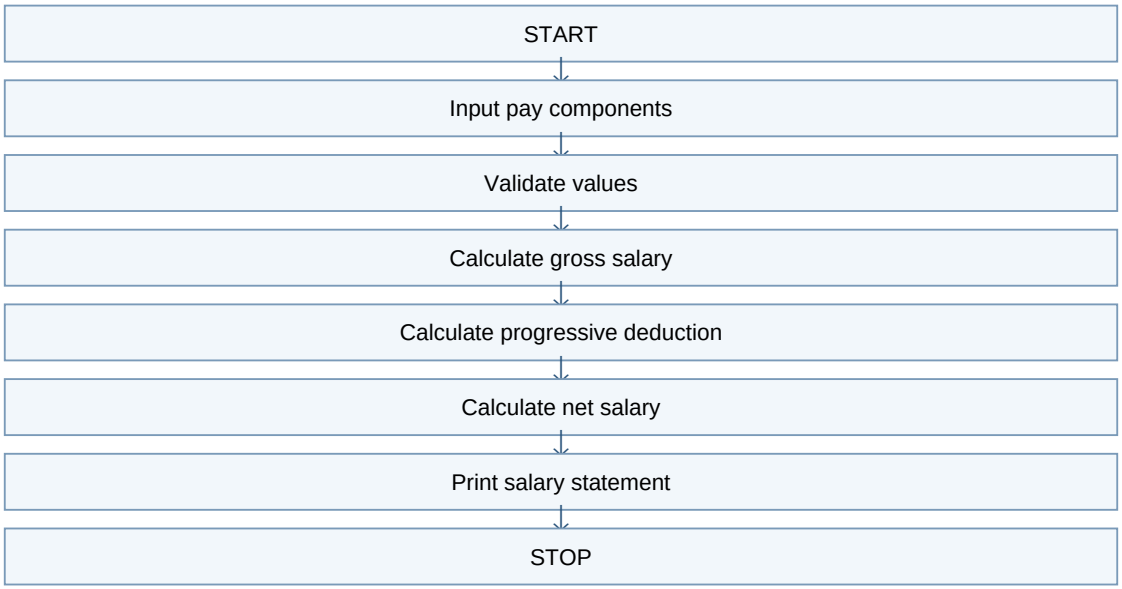
4. Calculate gross salary as the sum of all components.

5. Calculate progressive deductions: 5% of the first ■20,000; 10% of the portion from ■20,000 to ■50,000; and 15% of the portion above ■50,000.

6. Calculate net salary as gross salary minus deductions.

7. Display a formatted salary statement and stop.

Flowchart



Pseudocode

```
BEGIN
    INPUT basic, allowances, overtime, incentives
    IF any component < 0 THEN
        DISPLAY "Invalid salary details"
    ELSE
        gross = basic + allowances + overtime + incentives
        IF gross <= 20000 THEN
            deduction = gross * 0.05
        ELSE IF gross <= 50000 THEN
            deduction = 20000 * 0.05 + (gross - 20000) * 0.10
        ELSE
            deduction = 20000 * 0.05 + 30000 * 0.10
                        + (gross - 50000) * 0.15
        ENDIF
        net_salary = gross - deduction
        DISPLAY gross, deduction, net_salary
    ENDIF
END
```

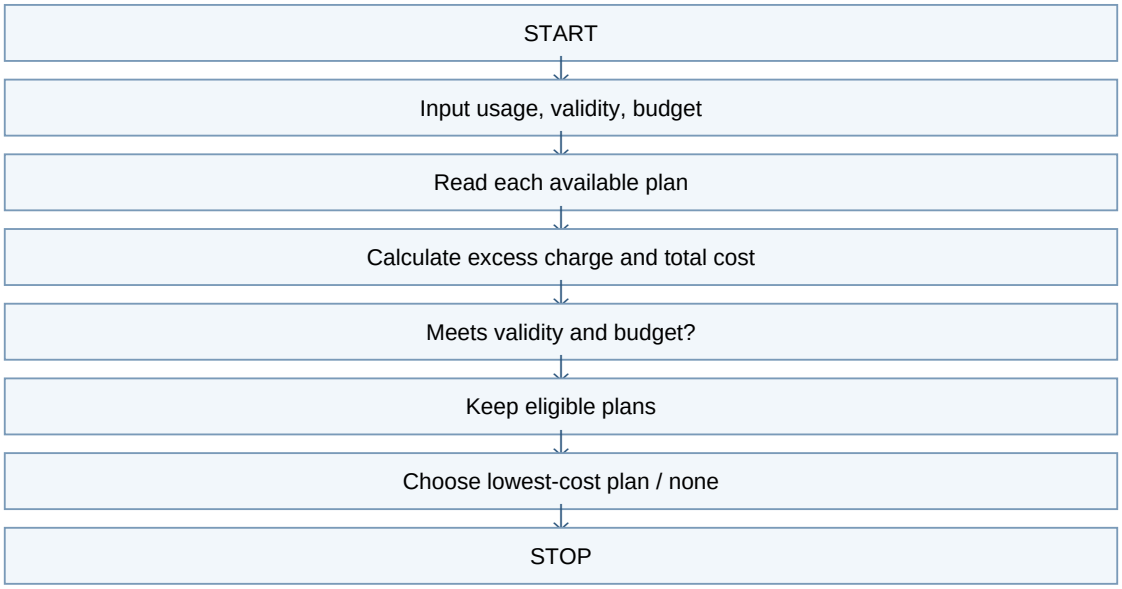
5. Multi-Plan Telecommunications Cost Optimization

Aim: Compare mobile plans against data usage, validity, and budget; select the lowest-cost eligible plan.

Algorithm

1. Start.
2. Input expected data usage, required validity, and budget.
3. Define plans with price, data allowance, and validity.
4. For each plan, calculate excess data and extra charges at ■50 per excess GB; calculate total cost.
5. Keep plans whose validity meets the requirement and total cost is within budget.
6. Choose the eligible plan with the lowest total cost.
7. Display the plan and price, or report that no plan qualifies. Stop.

Flowchart



Pseudocode

```
BEGIN
    INPUT usage_GB, required_days, budget
    plans = [(Basic,199,2,28),
             (Smart,299,3,56),
             (Premium,499,5,84)]
    eligible = empty list

    FOR each plan IN plans
        excess = MAX(0, usage_GB - plan.data_GB)
        total = plan.price + excess * 50
        IF plan.validity >= required_days AND total <= budget THEN
            ADD (total, plan.name) TO eligible
        ENDIF
    ENDFOR

    IF eligible is not empty THEN
        SELECT entry with minimum total cost
        DISPLAY plan name and total cost
    ELSE
        DISPLAY "No eligible plan"
    ENDIF
END
```

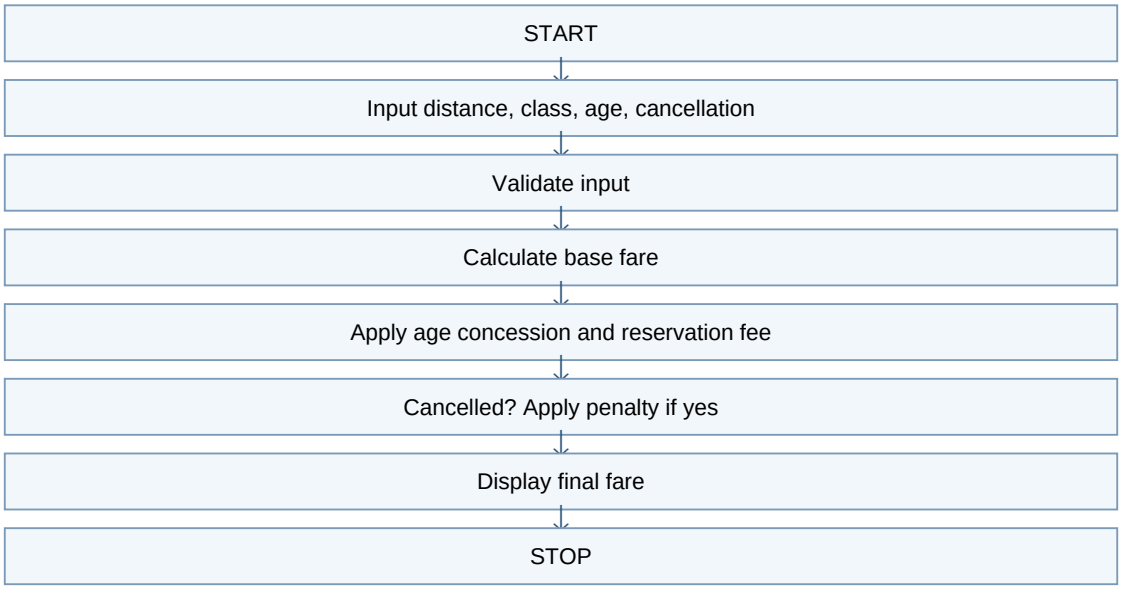
6. Railway Fare and Reservation Calculator

Aim: Calculate passenger fares using distance, travel class, age concession, reservation fee, and cancellation penalty.

Algorithm

1. Start.
2. Input distance, travel class, passenger age, and cancellation choice.
3. Validate distance and age; ensure the class is Sleeper or AC.
4. Calculate base fare at ■1/km for Sleeper or ■1.80/km for AC.
5. Apply a sample 20% base-fare concession for passengers aged 60 or above.
6. Add reservation fee: ■40 for Sleeper or ■80 for AC.
7. If cancelled, deduct a penalty equal to 25% of the calculated fare.
8. Display the fare breakdown and stop.

Flowchart



Pseudocode

```
BEGIN
    INPUT distance, class, age, cancelled
    IF distance <= 0 OR age < 0 OR class is invalid THEN
        DISPLAY "Invalid input"
    ELSE
        IF class = "sleeper" THEN
            rate = 1.00
            reservation = 40
        ELSE
            rate = 1.80
            reservation = 80
        ENDIF
        base = distance * rate
        IF age >= 60 THEN concession = base * 0.20
        ELSE concession = 0
        ENDIF
        fare = base - concession + reservation
        IF cancelled = "yes" THEN
            penalty = fare * 0.25
            fare = fare - penalty
        ENDIF
        DISPLAY base, concession, reservation, fare
    ENDIF
END
```

7. Retail Billing with Multiple Discount Rules

Aim: Create an itemized invoice with quantity, category-based taxes, threshold discounts, and a maximum discount limit.

Algorithm

1. Start.

2. Input the number of products.

3. For each product, input name, unit price, quantity, and category; validate price and quantity.

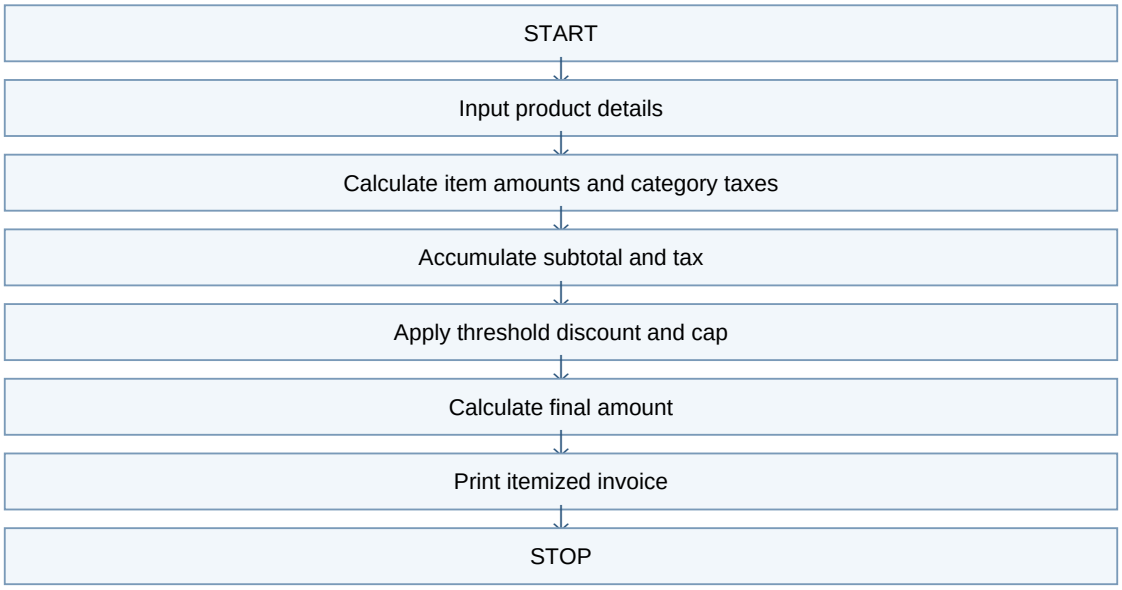
4. Calculate each item amount and tax: Food 5%, Electronics 18%, Other 12%.

5. Add item amounts and taxes to their respective totals.

6. Apply 10% discount if subtotal is at least ■5,000; otherwise 5% if subtotal is at least ■2,000; otherwise no discount. Cap discount at ■1,000.

7. Calculate final amount as subtotal minus discount plus tax. Print the invoice and stop.

Flowchart



Pseudocode

```
BEGIN
    INPUT number_of_products
    subtotal = 0
    tax_total = 0

    FOR each product
        INPUT name, price, quantity, category
        IF price < 0 OR quantity <= 0 THEN
            DISPLAY "Invalid item"
            CONTINUE
        ENDIF
        amount = price * quantity
        IF category = "food" THEN tax_rate = 0.05
        ELSE IF category = "electronics" THEN tax_rate = 0.18
        ELSE tax_rate = 0.12
        ENDIF
        subtotal = subtotal + amount
        tax_total = tax_total + amount * tax_rate
    ENDFOR

    IF subtotal >= 5000 THEN discount = subtotal * 0.10
    ELSE IF subtotal >= 2000 THEN discount = subtotal * 0.05
    ELSE discount = 0
    ENDIF
```

```
    discount = MIN(discount, 1000)
    final_amount = subtotal - discount + tax_total
    DISPLAY subtotal, discount, tax_total, final_amount
END
```

8. Manufacturing Cost and Profitability Analysis

Aim: Calculate material cost, wastage, labour, machine cost, revenue, and whether the target profit percentage is achieved.

Algorithm

1. Start.

2. Input production quantity, material cost per unit, wastage percentage, total labour cost, machine cost, selling price per unit, and target profit percentage.

3. Validate quantity and cost/percentage values.

4. Calculate material cost and wastage cost.

5. Calculate total production cost and sales revenue.

6. Calculate profit or loss and profit percentage.

7. Compare profit percentage with the target; display the result and stop.

Flowchart

Pseudocode

```
BEGIN
    INPUT quantity, material_rate, wastage_pct
    INPUT labour, machine, selling_price, target_pct

    IF inputs are invalid THEN
        DISPLAY "Invalid input"
    ELSE
        material = quantity * material_rate
        wastage_cost = material * wastage_pct / 100
        total_cost = material + wastage_cost + labour + machine
        revenue = quantity * selling_price
        profit = revenue - total_cost

        IF total_cost > 0 THEN
            profit_pct = profit / total_cost * 100
        ELSE
            profit_pct = 0
        ENDIF

        IF profit_pct >= target_pct THEN
            DISPLAY "Target achieved"
        ELSE
            DISPLAY "Target not achieved"
        ENDIF
```

```
        DISPLAY total_cost, revenue, profit, profit_pct
    ENDIF
END
```

9. Multi-Currency Travel Budget Analysis

Aim: Convert foreign-currency expenses to INR, include conversion fees, check budget, and identify the largest expense category.

Algorithm

1. Start.

2. Input travel budget in INR and the number of expense categories.

3. For each category, input name, foreign-currency amount, exchange rate in INR per currency unit, and conversion fee percentage.

4. Validate the amount, rate, and fee.

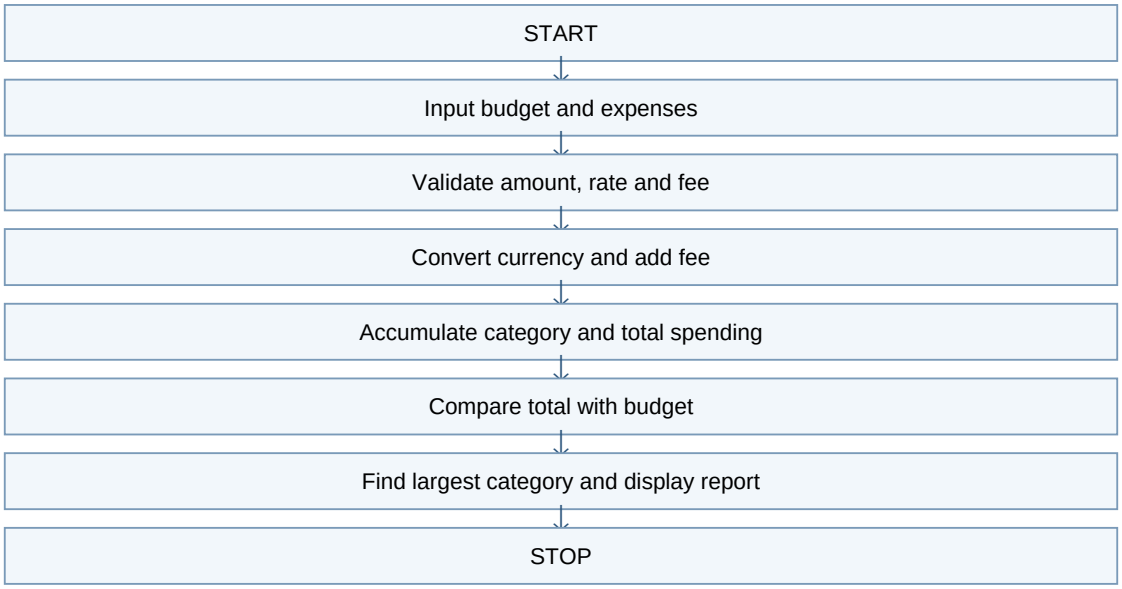
5. Calculate converted amount, conversion fee, and final INR amount.

6. Accumulate total spending by category and overall.

7. Compare spending with budget and identify the category with the largest total expense.

8. Display the report and stop.

Flowchart



Pseudocode

```
BEGIN
    INPUT budget_INR, number_of_categories
    total = 0
    category_totals = empty map

    FOR each expense
        INPUT category, amount, exchange_rate, fee_pct
        IF amount < 0 OR exchange_rate <= 0
           OR fee_pct < 0 OR fee_pct > 100 THEN
            DISPLAY "Invalid expense"
            CONTINUE
        ENDIF
        converted = amount * exchange_rate
        fee = converted * fee_pct / 100
        final_INR = converted + fee
        category_totals[category] += final_INR
        total += final_INR
    ENDFOR

    IF total > budget_INR THEN
        DISPLAY "Over budget", total - budget_INR
    ELSE
```

```
        DISPLAY "Remaining budget", budget_INR - total
    ENDIF

    IF category_totals is not empty THEN
        DISPLAY category with maximum category total
    ENDIF
END
```

10. Campus Event Financial Planning

Aim: Calculate event expenses, sponsorship shortfall, participant costs, and minimum registrations needed to break even.

Algorithm

1. Start.

2. Input venue, equipment, and transport costs; catering and certificate costs per participant; sponsorship, registration fee, and expected participants.

3. Validate all inputs.

4. Calculate fixed costs as venue + equipment + transport.

5. Calculate total event cost as fixed costs + (catering + certificate cost) × expected participants.

6. Calculate the remaining funding shortfall after sponsorship.

7. Calculate participant contribution as registration fee minus variable cost per participant. If positive, calculate minimum registrations as the ceiling of max(0, fixed costs - sponsorship) divided by participant contribution.

8. Compare expected participants with the break-even count and display the report. Stop.

Flowchart

Pseudocode

```
BEGIN
    INPUT venue, equipment, transport
    INPUT catering_per_person, certificate_per_person
    INPUT sponsorship, registration_fee, expected_participants

    IF any input is negative THEN
        DISPLAY "Invalid input"
    ELSE
        fixed_cost = venue + equipment + transport
        variable_cost = catering_per_person + certificate_per_person
        total_cost = fixed_cost + variable_cost * expected_participants
        sponsorship_used = MIN(sponsorship, total_cost)
        shortfall = MAX(0, total_cost - sponsorship_used)
        contribution = registration_fee - variable_cost

        IF contribution > 0 THEN
            remaining_fixed = MAX(0, fixed_cost - sponsorship)
            minimum_registrations =
                CEILING(remaining_fixed / contribution)
            DISPLAY minimum_registrations
        ELSE
```

```
            DISPLAY "Registration fee must exceed variable cost"
        ENDIF

        DISPLAY total_cost, sponsorship, shortfall
    ENDIF
END
```